hello abstract

1 Setup and calibration

Let $\mathscr{H} = \{h_1, \ldots, h_N\}$ be the discrete set of candidate classes. Our aim is to build a function C that maps an input x to a subset $C(x) \subseteq \mathscr{H}$ of classes, such that the true class lies in the set with probability at least $1-\alpha$. Our input is described by scores from several predictors. For each candidate class h we have a raw score vector $s_h = (s_{h,1}, \ldots, s_{h,K_h}) \in ([0,1] \cup \{\mathrm{NA}\})^{K_h}$, where $s_{h,k}$ is the evidence of the k-th predictor in favour of class h, while NA means that this predictor gave no score for the given class. We do not impute missing scores (NA). Instead, we record the set of observed coordinates $m_h = \{k : s_{h,k} \neq \mathrm{NA}\}$ and define all scoring functions on the pairs (s_h, m_h) and not on s_h alone. A score close to 1 means strong evidence for h, thus we map every observed coordinate to the nonconformity score $v_{h,k} = 1 - s_{h,k}$. When h is the true class, the vector v_h concentrates near the origin, while if h is not the true class, the vector v_h is further away. In Section 2 we write v for such a vector v_h and drop the index h.

Suppose we have a labelled calibration sample $\{(x_i, y_i)\}$. The predictors that produce the scores are trained on data that is disjoint from this sample and the test points. For each class h, the points with $y_i = h$ are randomly split into the shape discovery set $S_{1,h}$ and the size scaling set $S_{2,h}$. Each pair $(v_h(x_i), m_h(x_i))$ shows the score pattern for class h when it is actually the true class. A separate envelope is fitted for every class, using only the samples of that class.

Using $S_{1,h}$, we construct the scalar score $\tau_h(v,m)$, using one of the three envelope construction techniques in Section 2. We then compute the class specific scale factor $\hat{t}_h$ on $S_{2,h}$ as:

$$\hat{t}_h = \tau_{h,(k_h)}, \qquad k_h = \lceil (|S_{2,h}| + 1)(1-\alpha) \rceil, \tag{1}$$

where $\tau_{h,(k)}$ is the k-th smallest value of $\{\tau_h(v,m) : (v,m) \in S_{2,h}\}$, and $\hat{t}_h = \infty$ if $k_h > |S_{2,h}|$. The threshold value $\hat{t}_h$ is can be computed directly since the calibration set of class h only contains samples of this class. The prediction set of a new input x contains every class whose scaled envelope accepts the scores of x,

$$C(x) = \big\{ h \in \mathscr{H} : \ \tau_h\big(v_h(x), m_h(x)\big) \leq \hat{t}_h \big\}. \tag{2}$$

Theorem 1.1 (Class-conditional coverage)**.** *Assume that for every class h, the pairs (v_h, m_h) of the samples in $S_{1,h}$ and $S_{2,h}$ and of a new sample with true class h are exchangeable. Then, for every $h \in \mathscr{H}$,*

$$\mathbb{P}\big(Y \in C(X) \mid Y = h\big) \ \geq \ 1-\alpha,$$

and in particular $\mathbb{P}(Y \in C(X)) \geq 1-\alpha$. If in addition the values of τ_h are almost surely distinct, then $\mathbb{P}(Y \in C(X) \mid Y = h) \leq 1 - \alpha + 1/(|S_{2,h}|+1)$.

Proof. Let us fix a class h and write $n = |S_{2,h}|$ and $k = k_h$. We condition on $S_{1,h}$ and on the random directions of the radial method, which are independent of $S_{2,h}$ and of the new sample. Then τ_h is a fixed function. Let $T_1, \ldots, T_n$ be its values on $S_{2,h}$ and let T_0 be its value on a new sample with $Y = h$. Since $T_0, \ldots, T_n$ are exchangeable, the rank of T_0 is uniformly distributed over $\{1, \ldots, n+1\}$ when there are no ties, and ties can only make the rank of T_0 smaller. The event $\{h \in C(X)\}$ is $\{T_0 \leq T_{(k)}\}$, so

$$\mathbb{P}\big(T_0 \leq T_{(k)}\big) \ \geq \ \frac{k}{n+1} \ \geq \ 1-\alpha.$$

If $k > n$ then $\hat{t}_h = \infty$ and the probability is 1. If the values are almost surely distinct, the probability equals $k/(n+1) < 1 - \alpha + 1/(n+1)$. Averaging over $S_{1,h}$ and the directions gives the class conditional statement, and averaging over h with the class probabilities gives the marginal one. □

This is the Mondrian form of conformal prediction [**vovk2005algorithmic**]. It does not give coverage conditional on an individual input x, which is not possible in finite samples without strong assumptions. The observed sets m_h are part of the exchangeable pairs, so a new sample must have the same missingness mechanism as the calibration samples. If a class has few calibration samples, so that $k_h > |S_{2,h}|$, its threshold is infinite and it is always accepted. The library can also return the class with the smallest normalized score $\tau_h/\hat{t}_h$ when $C(x)$ is empty. This only enlarges the set, so the lower bound still holds, but we do not use this option in our experiments, which means that empty sets count as errors.

2 Envelope constructions

Each of the three envelope construction methods follows the same two stage calibration procedure: the shape-discovery set S_1 is used to determine the geometric shape of the envelope, then the size scaling set S_2 is used to determine the scaling factor $\hat{t}$. Each construction assigns a scalar score, $\tau(v,m)$, to a score vector v with observed set m. A candidate class is then included in the prediction set when $\tau(v,m) \le \hat{t}$, where $\hat{t}$ is calculated on S_2 as in Section 1. For both the collapsed and radial methods, we define $\tau(v) = \inf\{t > 0 : v \in t \cdot E\}$ using an envelope E which is star shaped with respect to the origin and give a closed form expression for it. The strip method works in the same way, except that the single scaling factor $\hat{t}$ multiplies the limits of the staircase rather than dilating the whole region, so τ is defined directly as a maximum of ratios.

We first fix a class h and drop the index h. K denotes the number of coordinates that remain after removing the coordinates that are missing for every sample of S_1. For a vector $v \in [0,\infty)^K$ and observed set $m \subseteq \{1,\dots,K\}$, we write v_j, $j \in m$, for just its observed entries. We use $\varepsilon = 10^{-12}$ to prevent division by zero, and $Q_p(\cdot)$ denotes the empirical p-quantile.

Collapsed envelope. The basic envelope construction simply collapses the K-dimensional score vector to a scalar score by taking the mean over the observed coordinates, and calibrates a single quantile on S_1:

$$\bar{v} = \frac{1}{|m|}\sum_{j\in m} v_j, \qquad \tilde{q} = Q_{1-\alpha}\big(\{\bar{v} : v \in S_1\}\big), \qquad \tau(v,m) = \frac{\bar{v}}{\tilde{q}}, \qquad E = \{v : \bar{v} \le \tilde{q}\}. \tag{3}$$

This method discards the geometric information that the K coordinates provide. Two vectors with the same mean are treated identically whatever the distribution of their coordinates is, so a single coordinate with strong evidence (a very small v_j) is averaged together with all the others. The accepted region is a halfspace in the observed coordinates. It serves as a mean score baseline for the multidimensional constructions below. After calibration on S_2, a vector is accepted when $\bar{v}/\tilde{q} \le \hat{t}$, which is the same as $\bar{v} \le \hat{t}\,\tilde{q}$. Here $\hat{t}$ is the k-th smallest value of $\bar{v}/\tilde{q}$ over $v \in S_2$, so $\hat{t}\,\tilde{q}$ is just the k-th smallest value of of $\bar{v}$ over $v \in S_2$.

Radial envelope. The radial envelope method retains the directional information of the scores. It works by calibrating a boundary radius as a function of direction on the unit sphere, which is restricted to the positive orthant since all nonconformity scores are nonnegative. We sample M directions $\{u_r\}_{r=1}^M$ uniformly from the positive orthant by drawing independent vectors $g_r \sim |\mathcal{N}(0, I_K)|$ and normalizing them, $u_r = g_r/\|g_r\|$. For each vector $v \in S_1$, we compute its norm $\|v\|$ and unit direction $d_v = v/\|v\|$ using only the observed coordinates. For each direction u_r, we collect the n_r points of S_1 within an angular sector of half width

$\delta = 30^\circ$, that is, those with $d_v \cdot u_r \geq \cos\delta$. The directional radius $\tilde{q}_r$ is then calibrated as

$$\tilde{q}_r = \begin{cases} Q_{1-\alpha}\big(\{\|v\| : d_v \cdot u_r \geq \cos\delta\}\big) & \text{if } n_r \geq n_{\min}, \\ w_r\, Q_{1-\alpha}\big(\{\|v\| : d_v \cdot u_r \geq \cos\delta\}\big) + (1-w_r)\,\tilde{q}_{\text{marg}} & \text{if } 0 < n_r < n_{\min}, \\ \tilde{q}_{\text{marg}} & \text{if } n_r = 0, \end{cases} \tag{4}$$

where $n_{\min} = \max\{10, \lfloor 0.2\,|S_1| \rfloor\}$ is the minimum number of points required, $w_r = n_r/n_{\min}$, and $\tilde{q}_{\text{marg}} = Q_{1-\alpha}(\{\|s\| : s \in S_1\})$ is the marginal fallback radius. This helps us avoid a hard cutoff between sufficient and insufficient local support. To get a smooth boundary, we compute the radius in an arbitrary direction d as a weighted average of all M calibrated radii,

$$R(d) = \sum_{r=1}^{M} \omega_r(d)\,\tilde{q}_r, \qquad \omega_r(d) = \frac{\exp(\kappa\, d \cdot u_r)}{\sum_{r'=1}^{M} \exp(\kappa\, d \cdot u_{r'})}, \tag{5}$$

where $\kappa > 0$ controls the angular smoothness (a larger κ gets closer to nearest-direction assignment). We evaluate R at the directions $a_p \in \{u_1, \ldots, u_M, e_1, \ldots, e_K\}$, where the e_j are the coordinate axes. This gives us the boundary points $b_p = R(a_p)\,a_p$. The envelope E is the union of the boxes $\{v : 0 \leq v \leq b_p\}$ over all p, and the scaling function has the closed form:

$$\tau(v, m) = \min_{p} \max_{j \in m} \frac{v_j}{b_{pj} + \varepsilon}, \tag{6}$$

which is $\inf\{t > 0 : v \in t \cdot E\}$ computed on the observed coordinates. We use $M = 250$ and $\kappa = 8$ by default.

$\to \to$ (twice, in the limits equation)

Strip envelope. The strip envelope works by defining conditional bounds for each target coordinate i based on the bin occupied by another coordinate j. We partition the range of the observed S_1 values of coordinate j into NB equal width bins, and write $\beta_j(x)$ for the bin that contains x (a value below the smallest or above the largest observed S_1 value of coordinate j is put in the first or last bin, respectively). We work with the level $\alpha' = \alpha/K$, a Bonferroni type correction for the K coordinates. It only affects the shape of the envelope, since the coverage is controlled by $\hat{t}$. For each bin n containing $n_{j,n}$ points of S_1 with both v_j and v_i observed, the upper limit along coordinate $i \neq j$ is calibrated as

$$L_{j,i,n} = \begin{cases} Q_{1-\alpha'}\big(\{v_i : v \in S_1,\ \beta_j(v_j) = n\}\big) & \text{if } n_{j,n} \geq n_0, \\ w_{j,n}\, Q_{1-\alpha'}\big(\{v_i : v \in S_1,\ \beta_j(v_j) = n\}\big) + (1-w_{j,n})\,\tilde{q}_i & \text{if } 0 < n_{j,n} < n_0, \\ \tilde{q}_i & \text{if } n_{j,n} = 0, \end{cases} \tag{7}$$

where $\tilde{q}_i$ is the marginal $(1-\alpha')$ quantile of coordinate i over S_1, n_0 is the minimum number of points required in a bin, and $w_{j,n} = n_{j,n}/n_0$ shrinks the limit of underpopulated bins towards it, which is similar to the blending rule of the radial envelope. Then there is a monotonicity pass that makes sure that the limits are nonincreasing in the bin index,

$$L_{j,i,n} \leftarrow \max_{n' \geq n} L_{j,i,n'}. \tag{8}$$

This ensures that a vector with a smaller value (more confident) in the conditioning coordinate j never receives a tighter bound on coordinate i than a vector with a larger value in j. The envelope E is a coordinatewise monotone staircase, which is the intersection of these pairwise constraints. If a coordinate

is unobserved, we ignore all the pairwise constraints involving it, so it is neither used as a target nor as a conditioning coordinate. The scaling function is

$$\tau(v,m) = \begin{cases} \max\limits_{\substack{j,i\in m \\ j\neq i}} \dfrac{v_i}{L_{j,i,\beta_j(v_j)}+\varepsilon}, & |m| \geq 2, \\ \dfrac{v_j}{\tilde{q}_j+\varepsilon}, & m=\{j\}, \end{cases} \tag{9}$$

so a vector is accepted when $v_i \leq \hat{t} L_{j,i,\beta_j(v_j)}$ for every pair of observed coordinates. The single scalar $\hat{t}$ multiplies all the limits, while the bins stay fixed. We use $NB = 8$ and $n_0 = 3$ by default.

Properties and missing scores. The three scaling functions share one structural property.

Proposition 2.1. *Fix an observed set m and let v, v' be score vectors with observed set m such that $v'_j \leq v_j$ for all $j \in m$. Then $\tau(v',m) \leq \tau(v,m)$ for each of the three constructions. In particular, every accepted region $\{\tau \leq \hat{t}\}$ is downward closed, so if a candidate class is accepted, it stays accepted when its scores improve.*

Proof. For (3) this is clear, because $\tilde{q}$ does not depend on v and the mean is nondecreasing in each coordinate. In (6) each ratio $v_j/(b_{pj}+\varepsilon)$ is nondecreasing in v_j, and a maximum and a minimum of nondecreasing functions is nondecreasing. For (9), $v'_j \leq v_j$ gives $\beta_j(v'_j) \leq \beta_j(v_j)$ because β_j is nondecreasing, so $L_{j,i,\beta_j(v'_j)} \geq L_{j,i,\beta_j(v_j)}$ by (8). Together with $v'_i \leq v_i$, every ratio in (9) gets smaller. The case $|m| = 1$ is again a ratio that is nondecreasing in v_j. □

The proposition is not needed for the coverage statement of Section 1. For fixed S_1 (and fixed directions u_r, which are drawn independently of S_2 and of the new observation), each τ is a fixed function of (v,m), so the same rank argument applies to all three. Monotonicity is only a sanity property of the learned shapes.

For a vector with observed set m, the collapsed method averages over m, and the radial and strip methods only use the coordinates in m. If no coordinate is observed, the collapsed method rejects the candidate class, while the radial and strip methods accept it. During fitting we require every vector in S_1 and S_2 to have at least one observed coordinate, so the calibration does not cover this case and we make no coverage claim for it.

3 Software

The constructions of Section 2 are implemented in the Python package `conformal_envelopes` [**conformalenvelopes**]. It depends only on NumPy, pandas and Matplotlib, and it is installed from the repository with `pip install -e .`. The package does not train any classifier. It takes the scores of predictors that were trained on separate data beforehand (as required in Section 1).

The data are a table (a pandas data frame or a CSV file) with one row per sample, an identifier column, a label column (needed only for fitting) and one column for each score. By default the scores lie in $[0,1]$; a higher score means better agreement with the class, and a missing score is NaN. The library converts the scores to the nonconformity scores $v = 1 - s$ of Section 1. All classes use the same score columns by default. If the predictors give a separate score for each candidate class, as in the example below, a dictionary `label_to_columns` lists the columns of each class.

We illustrate the usage with a shor Iris example of the repository. Two classifiers, trained on a separate part of the data, give one score for each species, and the radial envelope is fitted on a conformal sample.

We illustrate the usage on the Iris data, using two classifiers trained on a separate part of the data, which give one score for each species, and the radial envelope is fitted on a conformal sample. The notebook in the repository contains the same steps, with additional checks and plots.

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from conformal_envelopes import ConformalSetModel, evaluate_prediction_sets

iris = load_iris(as_frame=True)
X, y = iris.data, iris.target_names[iris.target]
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=30, stratify=y, random_state=0)
X_train, X_cal, y_train, y_cal = train_test_split(
    X_rest, y_rest, test_size=60, stratify=y_rest, random_state=1)

# Two classifiers give one score per candidate species.
clfs = {"logreg": LogisticRegression(max_iter=2000),
        "knn": KNeighborsClassifier()}
for clf in clfs.values():
    clf.fit(X_train, y_train)

def score_table(features):
    table = pd.DataFrame({"ID": features.index.astype(str)})
    for name, clf in clfs.items():
        for k, species in enumerate(clf.classes_):
            table[f"{species}_{name}"] = clf.predict_proba(features)[:, k]
    return table

cal = score_table(X_cal)
cal["label"] = y_cal
test = score_table(X_test)
cols = {s: [f"{s}_logreg", f"{s}_knn"] for s in iris.target_names}

model = ConformalSetModel(method="radial", alpha=0.10, random_state=0)
model.fit(cal, id_col="ID", label_col="label",
          score_cols=[c for v in cols.values() for c in v],
          label_to_columns=cols)
pred = model.predict(test)
print(evaluate_prediction_sets(pred["prediction_set"], list(y_test)))
```

The model is created with the method (`"collapsed"`, `"radial"` or `"strip"`), the level α and the optional parameters of the method from Section 2. `fit` splits the sample of each class into S_1 and S_2 (a fraction `shape_fraction`, 0.5 by default, goes to S_1), builds the envelope and computes $\hat{t}$. `predict` returns one row for each sample, with the prediction set, its size, the value of τ for each label and an indicator `forced`. The optional rule that replaces an empty set by the label with the smallest τ is on by default (`force_nonempty`), and it is switched off in Section 4. `evaluate_prediction_sets` computes the coverage, the average set size, the singleton rate, the empty set rate and the coverage of each class. Finally, a fitted model can be saved and loaded.

Visualisation. `plot` fits a separate two dimensional envelope for two selected predictors and shows it with the test samples, as in Figure **??**. `plot_slice` shows a cross section of the fitted envelope, with the other

coordinates fixed at chosen values. Finally, for each true class `plot_inclusion_heatmap` shows how often every label appears in the prediction sets, and its diagonal gives the coverage of each class.

Examples and tests. The repository contains notebooks for the Iris tutorial above, the simulated experiment and the two football experiments of Section 4. Since 30 test flowers are too few to judge the coverage, the Iris example only illustrates the use of the library.

4 Experiments

We evaluated the three envelope construction methods on simulated data with missing scores and on two football classification tasks. We keep $\alpha = 0.1$ throughout, the sample of each class is split equally into the shape discovery set S_1 and the size scaling set S_2, finally coverage is the proportion of test samples whose prediction set contains the true label. We report coverage, the average set size, the singleton rate and the empty-set rate, as mean $\pm$ standard error over repeated runs. The radial method uses $M = 100$, $\kappa = 4$ and $\delta = 30^\circ$, and the strip method uses $NB = 8$ with $n_0 = 3$ on the simulated data and $n_0 = 10$ on the football data; all other parameters are the default parameters in the library. The code and the saved outputs are available at [**conformalenvelopes**].

Simulated data. There are two classes, A and B, and four predictors which are used to score each candidate class. For a sample with true class h^* and a candidate class h, let $\sigma = +1$ if $h = h^*$ and $\sigma = -1$ otherwise. The first predictor has the score $1/(1 + e^{-(1.8\,\sigma + \varepsilon_1)})$ with $\varepsilon_1 \sim N(0,1)$. The second is built in the same way from a noise term ε_2 with correlation 0.9 to ε_1. The third is weak, it has a shift $0.35\,\sigma$ and noise of standard deviation 1.5, and the fourth is uniform on $[0,1]$, it carries no information. Each score is then independently hidden with probability 0.15. If all four scores of a candidate are hidden the draw is repeated. In each run there are 1500 training draws and 500 test samples, and we use 500 independent runs.

Football data. We classify FIFA 19 players from 34 skill ratings [**fifa19data**]. In the first task the label is simpler, it is just the role (defender, midfielder, attacker or goalkeeper) for 18,147 players. In the second it is the recorded position, in this we only use the 17,835 players in the 20 positions with at least 100 player records. Four classifiers (logistic regression, random forest, gradient boosting and a support vector classifier) are trained on a separate 40% of the players, and the scores are their predicted probabilities of each label. Another 20% is used for validation, and the remaining players are used for conformal fitting (30%) and testing (10%). To measure the variability of the coverage, we keep the trained classifiers fixed and evaluate them over 100 random split of the remaining data into calibration and test sets (sized 5,444 / 1,815 for roles and 5,351 / 1,784 for positions). The data contain no missing scores.

Results. Table 1 shows that the coverage of all three constructions is close to the target 0.9 in every experiment. The coverage of each class is also close to the target; it is between 0.899 and 0.902 on the simulated data, and between 0.896 and 0.902 for the four roles. For the positions, the average coverage of a position lies between 0.888 and 0.926, it has more range due to the smaller number of data points for each position.

The methods have similar set sizes on the football role tasks, where most of the sets are singletons. But on the position classification task the sets contain between 4.4 and 4.7 positions on average, which is expected, because neighbouring positions, such as the left, central and right variants of a role, have very similar skill values. The order of the methods by set size changes between the experiments, therefore we cannot claim

Data	Method	Coverage	Set size	Singleton	Empty
Simulated (500 runs)	Collapsed	0.9012 ± 0.0008	1.014	0.811	0.088
	Radial	0.9001 ± 0.0008	1.086	0.752	0.081
	Strip	0.8998 ± 0.0008	0.955	0.856	0.095
Football roles (100 runs)	Collapsed	0.9002 ± 0.0009	1.069	0.887	0.022
	Radial	0.8995 ± 0.0010	1.068	0.886	0.023
	Strip	0.8994 ± 0.0010	1.082	0.856	0.031
Football positions (100 runs)	Collapsed	0.9032 ± 0.0009	4.364	0.108	0.010
	Radial	0.9016 ± 0.0009	4.738	0.101	0.010
	Strip	0.9016 ± 0.0009	4.575	0.118	0.011

Table 1: Results as mean $\pm$ standard error over repeated runs, with target coverage 0.9.

that one construction is better in general. The football dataset is complete, so the handling of missing scores is only evaluated on the simulated data.